
FIUBA

Facultad de Ingeniería — Universidad de Buenos Aires

Redes (TA-048)

Trabajo Práctico N°2

SDN / NAT

INTEGRANTES

Franco Almada	Padrón 104498
Alfredo Godoy	Padrón 109518
Brian Gabriel Quispe	Padrón 109565
Esteban Davila	Padrón 109815

Junio 2026

Trabajo Práctico N°2 — SDN / NAT

Redes (TA-048) — FIUBA

1. Descripción General de la Solución

Se implementó un NAT saliente con traducción por puertos (PAT) sobre el controlador SDN POX, extendiendo el protorouter provisto por la cátedra. El sistema permite que múltiples hosts de la red privada (192.168.1.0/24) se comuniquen con la red pública (200.0.0.0/24) compartiendo una única IP pública, manteniendo múltiples conexiones simultáneas para TCP y UDP.

La arquitectura final se descompone en cuatro módulos:

Módulo	Archivo	Responsabilidad
Coordinador principal	protorouter.py	Orquesta los demás módulos; único listener de eventos OpenFlow
Manejo de ARP	arp_handler.py	Resolución dinámica de MACs, tabla ARP, cola de paquetes pendientes
Tabla NAT	nat_table.py	Estado de las traducciones activas; asignación de puertos públicos
Gestión de flujos	flow_manager.py	Construcción e instalación de flow_mod en el switch
Configuración	config.py	Parámetros centralizados (IPs, MACs, puertos, timeouts)

Topología

El switch OpenFlow actúa como dispositivo de capa 3 con dos "interfaces virtuales" manejadas por el controlador:

- **Interfaz pública:** IP 200.0.0.254, MAC 00:00:00:aa:aa:aa, puerto 1
- **Interfaz privada:** IP 192.168.1.254, MAC 00:00:00:bb:bb:bb, puertos 2-4

2. Diseño de la Tabla NAT

La tabla NAT mantiene dos índices complementarios para traducción bidireccional:

Tabla saliente (_nat_out)

- Clave: (protocolo, ip_privada, puerto_privado)
- Valor: puerto_público

Permite determinar, dado un paquete saliente de un host privado, qué puerto público usar.

Tabla entrante (_nat_in)

- Clave: (protocolo, puerto_público)
- Valor: (ip_privada, puerto_privado, puerto_switch)

Permite traducir un paquete entrante al host privado correcto, incluyendo el puerto físico del switch al que enviar.

Ejemplo conceptual

Protocolo	IP privada	Puerto privado	IP pública	Puerto público
TCP	192.168.1.2	54321	200.0.0.254	10000
TCP	192.168.1.3	54321	200.0.0.254	10001
UDP	192.168.1.2	5001	200.0.0.254	10002
UDP	192.168.1.3	5001	200.0.0.254	10003

Asignación de puertos públicos

El rango de puertos públicos disponibles se configura en config.py:

```
NAT_PORT_START = 10000 NAT_PORT_END = 65535
```

El allocator recorre el rango de forma circular (_next_port), saltando los puertos ya en uso (_used_ports). Cuando un flujo expira (evento FlowRemoved), el puerto se libera y queda disponible para reutilización.

3. Manejo de ARP

El módulo arp_handler.py implementa resolución ARP completamente dinámica, sin entradas estáticas en ningún host.

Tabla ARP interna

```
arp_table: { ip → (mac, puerto_switch) }
```

Se actualiza con cada paquete ARP recibido (aprendizaje pasivo desde el campo protosrc del ARP).

Flujo de resolución

1. Un host privado envía un ARP Request consultando la IP del gateway (192.168.1.254).
2. ARPHandler._handle_request() detecta que es para PRIVATE_IP y responde con PRIVATE_MAC.
3. El host privado aprende la MAC del gateway y envía el paquete IP.
4. El controlador recibe el paquete IP. Si no conoce la MAC del destino público, encola el paquete en _pending[dst_ip] y emite un ARP Request hacia la red pública.
5. h1 responde con su MAC. El controlador aprende la MAC, procesa todos los paquetes encolados y llama al callback _on_mac_resolved.
6. El callback reinyecta cada paquete pendiente en _handle_ip, que ahora sí puede completar la traducción e instalar los flujos.

Cola de paquetes pendientes

```
_pending: { ip_destino → [(in_port, raw_packet), ...] }
```

Evita perder paquetes que llegaron antes de que la MAC del destino fuera conocida. Solo se emite un ARP Request por IP desconocida (el primero); los siguientes paquetes simplemente se encolan.

4. Estrategia de Instalación de Flujos

Una vez resueltas todas las MACs y asignado el puerto público NAT, el controlador instala dos flujos OpenFlow en el switch por cada nueva conexión:

Flujo saliente

Coincide con paquetes cuyo origen es la IP/puerto privados y destino es la IP/puerto públicos. Acciones: Reescribir IP origen → PUBLIC_IP; Reescribir puerto origen → pub_port (TCP/UDP); Reescribir MAC origen → PUBLIC_MAC; Reescribir MAC destino → MAC del servidor; Enviar por PUBLIC_PORT (puerto 1 del switch).

Flujo entrante

Coincide con paquetes cuyo destino es PUBLIC_IP:pub_port entrando por PUBLIC_PORT. Acciones: Reescribir IP destino → IP privada; Reescribir puerto destino → puerto privado (TCP/UDP); Reescribir MAC origen → PRIVATE_MAC; Reescribir MAC destino → MAC del host privado; Enviar por el puerto del switch correspondiente al host privado.

Timeout y limpieza

Los flujos se instalan con `idle_timeout = 60 s` y el flag `OFPPF_SEND_FLOW_REM`. Cuando el switch expira un flujo por inactividad, notifica al controlador con un evento `FlowRemoved`, que identifica el flujo saliente por `nw_src == PUBLIC_IP` y libera la entrada NAT correspondiente.

Primer paquete

El primer paquete de cada conexión llega al controlador vía `PacketIn` (antes de que existan flujos). Luego de instalar los flujos, el controlador lo traduce y reenvía manualmente (`_forward_outbound`). A partir del segundo paquete, el switch lo procesa directamente sin intervención del controlador.

5. Decisiones de Diseño

Separación en módulos: se optó por dividir la lógica en `ARPHandler`, `NATTable` y `FlowManager`, cada uno con responsabilidad única. `ProtoRouter` actúa como coordinador y es el único que recibe eventos OpenFlow. Esto facilita la prueba y el mantenimiento de cada componente de forma independiente.

Sin hardcodeo de MACs o IPs de hosts: toda la información de los hosts se aprende dinámicamente mediante ARP. La única información fija es la de las interfaces del propio NAT (en `config.py`), lo que permite cambiar MACs o IPs de los hosts en tiempo de ejecución sin modificar el código.

Dirección inversa bloqueada: los paquetes que no provienen de la red privada ni están destinados a PUBLIC_IP son descartados silenciosamente. Esto impide que h1 inicie conexiones directas a hosts privados.

Reutilización de puertos: el allocator libera puertos al recibir FlowRemoved, permitiendo su reutilización. El recorrido circular del rango evita agotar puertos en escenarios de alta rotación.

6. Pruebas Realizadas

Las pruebas pueden ejecutarse de forma automatizada con `test_nat.py`, o manualmente abriendo terminales xterm para cada host desde la CLI de Mininet:

```
mininet> xterm h1 h2 h3
```

A continuación se detallan los comandos exactos para cada prueba manual.

6.1 TCP básico (HTTP con curl)

Verifica traducción TCP completa y respuesta HTTP 200.

xterm h1 — levantar servidor:

```
python3 -m http.server 8080
```

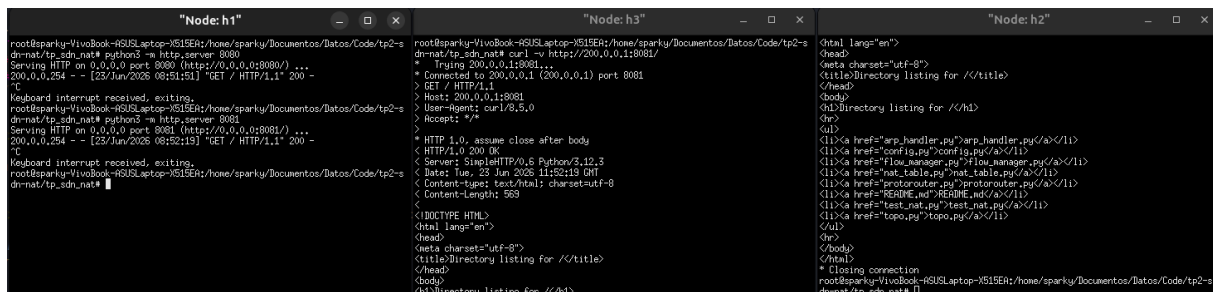
xterm h2 — hacer request:

```
curl -v http://200.0.0.1:8080/
```

xterm h3 — hacer request (puerto diferente para evitar colisión de flujos activos):

```
# En xterm h1, levantar otro servidor en otro puerto: python3 -m http.server 8081 # En
xterm h3: curl -v http://200.0.0.1:8081/
```

Resultado esperado: curl reporta HTTP/1.0 200 OK y muestra el listado de directorio.



```
"Node: h1"
root@sparky-VivoBook-4SUSLaptop-VS15ER:/home/sparky/Documentos/Batos/Code/tp2-s
drmat/tp_sdn_nat# python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
200.0.0.254 - - [25/Jun/2025 08:51:51] "GET / HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.
root@sparky-VivoBook-4SUSLaptop-VS15ER:/home/sparky/Documentos/Batos/Code/tp2-s
drmat/tp_sdn_nat# python3 -m http.server 8081
Serving HTTP on 0.0.0.0 port 8081 (http://0.0.0.0:8081/) ...
200.0.0.254 - - [25/Jun/2025 08:52:15] "GET / HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.
root@sparky-VivoBook-4SUSLaptop-VS15ER:/home/sparky/Documentos/Batos/Code/tp2-s
drmat/tp_sdn_nat#

"Node: h2"
root@sparky-VivoBook-4SUSLaptop-VS15ER:/home/sparky/Documentos/Batos/Code/tp2-s
drmat/tp_sdn_nat# curl -v http://200.0.0.1:8080/
* Trying 200.0.0.1:8080...
* Connected to 200.0.0.1 (200.0.0.1) port 8080
> GET / HTTP/1.1
> Host: 200.0.0.1:8080
> User-Agent: curl/8.5.0
> Accept: */*
* HTTP/1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.12.3
< Date: Tue, 25 Jun 2025 11:52:19 GMT
< Content-type: text/html; charset=utf-8
< Content-Length: 569
<
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<div>Directory listing for /</div>
</body>
</html>
* Closing connection
root@sparky-VivoBook-4SUSLaptop-VS15ER:/home/sparky/Documentos/Batos/Code/tp2-s
drmat/tp_sdn_nat#

"Node: h3"
root@sparky-VivoBook-4SUSLaptop-VS15ER:/home/sparky/Documentos/Batos/Code/tp2-s
drmat/tp_sdn_nat# curl -v http://200.0.0.1:8081/
* Trying 200.0.0.1:8081...
* Connected to 200.0.0.1 (200.0.0.1) port 8081
> GET / HTTP/1.1
> Host: 200.0.0.1:8081
> User-Agent: curl/8.5.0
> Accept: */*
* HTTP/1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.12.3
< Date: Tue, 25 Jun 2025 11:52:19 GMT
< Content-type: text/html; charset=utf-8
< Content-Length: 569
<
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<div>Directory listing for /</div>
</body>
</html>
* Closing connection
root@sparky-VivoBook-4SUSLaptop-VS15ER:/home/sparky/Documentos/Batos/Code/tp2-s
drmat/tp_sdn_nat#
```

Captura: 6.1 TCP básico (HTTP con curl)

6.2 TCP simultáneo

Verifica múltiples conexiones TCP activas al mismo tiempo. Se usa `iperf` en modo TCP con duración larga para garantizar solapamiento real.

xterm h1 — levantar servidor iperf TCP:

```
iperf -s -p 5010
```

xterm h2 (lanzar en background):

```
iperf -c 200.0.0.1 -p 5010 -t 15 > /tmp/tcp_h2.txt &
```

xterm h3 (lanzar inmediatamente después):

```
iperf -c 200.0.0.1 -p 5010 -t 15 > /tmp/tcp_h3.txt &
```

Mientras corren, verificar desde la **CLI de Mininet** que hay dos flujos TCP activos al mismo tiempo:

```
ovs-ofctl dump-flows s1
```

Deben aparecer entradas con `nw_src=192.168.1.2` y `nw_src=192.168.1.3` simultáneamente. Luego de ~20 segundos verificar:

xterm h2:

```
cat /tmp/tcp_h2.txt
```

xterm h3:

```
cat /tmp/tcp_h3.txt
```

Resultado esperado: ambos archivos muestran transferencia con Mbits/sec.

```
Connecting to remote controller at 1:
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> xterm h1 h2 h3
mininet> sh ovs-ofctl dump-flows s1
cookie=0x0, duration=2.305s, table=0, n_packets=230566, n_bytes=10199858568, idle_timeout=60, tcp,nw_src=192.168.1.2,nw_dst=200.0.0.1,tp_src=42972,tp_dst=5010 actions=mod_nw_src:200.0.0.254,mod_tp_src:10000,mod_dl_src:00:00:00:00:aa:aa,mod_dl_dst:00:00:00:00:01,output:"s1-eth1"
cookie=0x0, duration=1.443s, table=0, n_packets=118362, n_bytes=5200163320, idle_timeout=60, tcp,nw_src=192.168.1.3,nw_dst=200.0.0.1,tp_src=47102,tp_dst=5010 actions=mod_nw_src:200.0.0.254,mod_tp_src:10001,mod_dl_src:00:00:00:00:aa:aa,mod_dl_dst:00:00:00:00:01,output:"s1-eth1"
cookie=0x0, duration=2.264s, table=0, n_packets=147255, n_bytes=9718866, idle_timeout=60, tcp,in_port="s1-eth1",nw_dst=200.0.0.254,tp_dst=10000 actions=mod_nw_dst:192.168.1.2,mod_tp_dst:42972,mod_dl_src:00:00:00:bb:bb:bb,mod_dl_dst:00:00:00:00:02,output:"s1-eth2"
cookie=0x0, duration=1.482s, table=0, n_packets=73824, n_bytes=4872420, idle_timeout=60, tcp,in_port="s1-eth1",nw_dst=200.0.0.254,tp_dst=10001 actions=mod_nw_dst:192.168.1.3,mod_tp_dst:47102,mod_dl_src:00:00:00:bb:bb:bb,mod_dl_dst:00:00:00:00:03,output:"s1-eth3"
```

Captura: 6.2 TCP simultáneo

6.3 UDP básico (iperf)

Verifica traducción UDP y flujo de retorno (estadísticas iperf).

xterm h1 — levantar servidor UDP:

```
iperf -s -u -p 5001
```

xterm h2:

```
iperf -c 200.0.0.1 -u -p 5001 -t 4
```

Luego repetir con h3 en puerto diferente:

xterm h1 — segundo servidor:

```
iperf -s -u -p 5002
```

xterm h3:

```
iperf -c 200.0.0.1 -u -p 5002 -t 4
```

Resultado esperado: iperf reporta transferencia con bits/sec en ambos casos.

```
"Node: h1" "Node: h2" "Node: h3"
root@sparky-VivoBook-R515EH:/home/sparky/Documentos/Datos/Code/tp2-s drmat/tp_sdn_nat# iperf -s -u -p 5001
Server listening on UDP port 5001
UDP buffer size: 200 KByte (default)
[ 1] local 200.0.0.1 port 5001 connected with 200.0.0.254 port 10003
[ 10] Interval Transfer Bandwidth Jitter Lost/Total Datagrams
[ 1] 0.0000-4.0122 sec 517 KBytes 1.05 Mbits/sec 0.013 ms 0/360 (0%)
"Croot@sparky-VivoBook-R515EH:/home/sparky/Documentos/Datos/Code/tp2-s drmat/tp_sdn_nat# iperf -s -u -p 5002
Server listening on UDP port 5002
UDP buffer size: 200 KByte (default)
[ 1] local 200.0.0.1 port 5002 connected with 200.0.0.254 port 10004
[ 10] Interval Transfer Bandwidth Jitter Lost/Total Datagrams
[ 1] 0.0000-4.0132 sec 517 KBytes 1.05 Mbits/sec 0.012 ms 0/360 (0%)
root@sparky-VivoBook-R515EH:/home/sparky/Documentos/Datos/Code/tp2-s drmat/tp_sdn_nat#
root@sparky-VivoBook-R515EH:/home/sparky/Documentos/Datos/Code/tp2-s drmat/tp_sdn_nat# iperf -c 200.0.0.1 -u -p 5001 -t 4
Client connecting to 200.0.0.1, UDP port 5001
Sending 1470 byte datagrams, IPG target: 11215.21 us (kcalman adjust)
UDP buffer size: 200 KByte (default)
[ 1] local 192.168.1.2 port 5001 connected with 200.0.0.1 port 5001
[ 10] Interval Transfer Bandwidth Jitter Lost/Total Datagrams
[ 1] 0.0000-4.0152 sec 517 KBytes 1.05 Mbits/sec
[ 1] Sent 361 datagrams
[ 1] Server Report:
[ 10] Interval Transfer Bandwidth Jitter Lost/Total Datagrams
[ 1] 0.0000-4.0122 sec 517 KBytes 1.05 Mbits/sec 0.013 ms 0/360 (0%)
root@sparky-VivoBook-R515EH:/home/sparky/Documentos/Datos/Code/tp2-s drmat/tp_sdn_nat#
root@sparky-VivoBook-R515EH:/home/sparky/Documentos/Datos/Code/tp2-s drmat/tp_sdn_nat# iperf -c 200.0.0.1 -u -p 5002 -t 4
Client connecting to 200.0.0.1, UDP port 5002
Sending 1470 byte datagrams, IPG target: 11215.21 us (kcalman adjust)
UDP buffer size: 200 KByte (default)
[ 1] local 192.168.1.3 port 5002 connected with 200.0.0.1 port 5002
[ 10] Interval Transfer Bandwidth Jitter Lost/Total Datagrams
[ 1] 0.0000-4.0152 sec 517 KBytes 1.05 Mbits/sec
[ 1] Sent 361 datagrams
[ 1] Server Report:
[ 10] Interval Transfer Bandwidth Jitter Lost/Total Datagrams
[ 1] 0.0000-4.0132 sec 517 KBytes 1.05 Mbits/sec 0.012 ms 0/360 (0%)
root@sparky-VivoBook-R515EH:/home/sparky/Documentos/Datos/Code/tp2-s drmat/tp_sdn_nat#
```

Captura: 6.3 UDP básico (iperf)

6.4 UDP simultáneo

Verifica múltiples flujos UDP activos al mismo tiempo. Se aumenta la duración para garantizar solapamiento real.

xterm h1 — levantar servidor iperf UDP:

```
iperf -s -u -p 5003
```

xterm h2 (lanzar en background):

```
iperf -c 200.0.0.1 -u -p 5003 -t 15 > /tmp/udp_h2.txt &
```

xterm h3 (lanzar inmediatamente después):

```
iperf -c 200.0.0.1 -u -p 5003 -t 15 > /tmp/udp_h3.txt &
```

Mientras corren, verificar desde la **CLI de Mininet** que hay dos flujos UDP activos al mismo tiempo:

```
ovs-ofctl dump-flows s1
```

Deben aparecer entradas con `nw_src=192.168.1.2` y `nw_src=192.168.1.3` simultáneamente. Luego de ~20 segundos verificar:

xterm h2:

```
cat /tmp/udp_h2.txt
```

xterm h3:

```
cat /tmp/udp_h3.txt
```

Resultado esperado: ambos archivos muestran estadísticas de transferencia con bits/sec.

```
*** Adding control
Unable to connect
Connecting to remote host...
*** Adding hosts:
h1 h2 h3
*** Adding switch:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts:
h1 h2 h3
*** Starting controller:
c0
*** Starting 1 switch:
s1 ...
*** Starting CLI:
mininet> xterm h1 h2 h3
mininet> sh ovs-ofctl dump-flows s1
table=0, n_packets=380, n_bytes=574560, idle_timeout=60, udp,nw_src=192.168.1.2,nw_dst=200.0.0.1,tp_src=49363,tp_dst=5003 actions=mod_nw_src:200.0.0.254,mod_tp_s
rc:10000,mod_dl_src:00:00:00:aa:aa:aa,mod_dl_dst:00:00:00:00:00:01,output:"s1-eth1"
cookie=0x0, duration=3.930s, table=0, n_packets=317, n_bytes=479304, idle_timeout=60, udp,nw_src=192.168.1.3,nw_dst=200.0.0.1,tp_src=35195,tp_dst=5003 actions=mod_nw_src:200.0.0.254,mod_tp_s
rc:10001,mod_dl_src:00:00:00:aa:aa:aa,mod_dl_dst:00:00:00:00:00:01,output:"s1-eth1"
cookie=0x0, duration=4.589s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, udp,in_port="s1-eth1",nw_dst=200.0.0.254,tp_dst=10000 actions=mod_nw_dst:192.168.1.2,mod_tp_dst:49363,mod_dl_sr
c:00:00:00:bb:bb:bb,mod_dl_dst:00:00:00:00:00:02,output:"s1-eth2"
cookie=0x0, duration=3.889s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, udp,in_port="s1-eth1",nw_dst=200.0.0.254,tp_dst=10001 actions=mod_nw_dst:192.168.1.3,mod_tp_dst:35195,mod_dl_sr
c:00:00:00:bb:bb:bb,mod_dl_dst:00:00:00:00:00:03,output:"s1-eth3"
mininet>
```

Captura: 6.4 UDP simultáneo

6.5 NAT no hardcodedo (cambio de MAC en caliente)

Verifica que el controlador aprende MACs dinámicamente y no depende de valores fijos.

Nota: al bajar y subir la interfaz, Mininet pierde la ruta default (vía 192.168.1.254) pero no la restaura automáticamente. Es necesario agregarla a mano después de subir la interfaz. Esto no es un bug del NAT sino un comportamiento de Mininet.

xterm h1 — levantar servidor primero:

```
python3 -m http.server 8083
```

xterm h2 — cambiar MAC, restaurar ruta y verificar:

```
# Bajar interfaz, cambiar MAC, subir ip link set h2-eth0 down ip link set h2-eth0
address 00:00:00:00:ff:ff ip link set h2-eth0 up # Restaurar ruta default (se pierde al
bajar la interfaz en Mininet) ip route add default via 192.168.1.254 # Verificar MAC y
ruta ip link show h2-eth0 ip route show # Limpiar ARP cache ip neigh flush all # Probar
conectividad curl -v http://200.0.0.1:8083/
```

Resultado esperado: ip route show muestra default via 192.168.1.254. El curl retorna HTTP/1.0 200 OK — el controlador aprendió la nueva MAC 00:00:00:00:ff:ff vía ARP sin ninguna configuración adicional. Al terminar, restaurar:

xterm h2:

```
ip link set h2-eth0 down ip link set h2-eth0 address 00:00:00:00:00:02 ip link set
h2-eth0 up ip route add default via 192.168.1.254
```

```
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
DEBUG:core:POX 0.7.0 (gar) going up...
DEBUG:core:Running on CPython (3.12.3/Mar 23 2026 19:04:32)
DEBUG:core:Platform is Linux-6.17.0-35-generic-x86_64-with-glibc2.39
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.12.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
DEBUG:openflow.of_01:Listening on 0.0.0.0:6633
INFO:openflow.of_01:[00-00-00-00-00-01 2] connected
INFO:protorouter:Iniciando ProtoRouter para switch 1
INFO:protorouter:ProtoRouter (NAT/PAT) iniciado.
DEBUG:openflow.of_01:1 connection aborted
INFO:arp_handler:ARP aprendido: 192.168.1.2 → 00:00:00:00:ff:ff (port 2)
INFO:nat_table:NAT nueva entrada: 192.168.1.2:39260 → puerto público 10000
INFO:arp_handler:ARP REQUEST enviado: ¿quién tiene 200.0.0.1? (desde 200.0.0.254)
INFO:nat_table:NAT nueva entrada: 200.0.0.1 → 00:00:00:00:00:01 (port 1)
INFO:arp_handler:ARP REPLY recibido: 200.0.0.1 tiene 00:00:00:00:00:01
INFO:arp_handler:Procesando 1 paquete(s) pendiente(s) para 200.0.0.1
INFO:flow_manager:Flujo SALIENTE instalado: 192.168.1.2:39260 → 200.0.0.1:8083 (pub_port=10000)
INFO:flow_manager:Flujo ENTRANTE instalado: pub_port=10000 → 192.168.1.2:39260 (sw_port=2)
INFO:protorouter:ENVIADO: 200.0.0.254:10000 → 200.0.0.1:8083 | MAC: 00:00:00:aa:aa:aa → 00:00:00:00:01
INFO:arp_handler:ARP aprendido: 200.0.0.1 → 00:00:00:00:00:01 (port 1)
INFO:arp_handler:ARP REQUEST para IP pública (200.0.0.254), respondiendo con 00:00:00:aa:aa:aa
```

Captura: 6.5 NAT no hardcodedo (cambio de MAC en caliente)

6.6 Dirección inversa bloqueada (h1 → h2)

Verifica que el NAT **no** permite conexiones iniciadas desde la red pública.

xterm h2 — levantar servidor:

```
python3 -m http.server 9090
```

xterm h1 — intentar conectarse directamente a h2:

```
curl -v --max-time 5 http://192.168.1.2:9090/
```

Resultado esperado: curl agota el timeout sin respuesta (Connection timed out o Network is unreachable). **No** debe retornar HTTP 200.

```
"Node: h2"
root@sparky-VivoBook-ASUSLaptop-X515EA:/home/sparky/Documentos/Datos/Code/tp2-s
dn-nat/tp_sdn_nat# python3 -m http.server 9090
Serving HTTP on 0.0.0.0 port 9090 (http://0.0.0.0:9090/) ...

"Node: h1"
root@sparky-VivoBook-ASUSLaptop-X515EA:/home/sparky/Documentos/Datos/Code/tp2-s
dn-nat/tp_sdn_nat# curl -v --max-time 5 http://192.168.1.2:9090/
* Trying 192.168.1.2:9090...
* Connection timed out after 5002 milliseconds
* Closing connection
curl: (28) Connection timed out after 5002 milliseconds
root@sparky-VivoBook-ASUSLaptop-X515EA:/home/sparky/Documentos/Datos/Code/tp2-s
dn-nat/tp_sdn_nat#
```


6.7 Flujos instalados en el switch

Verifica que el switch instala reglas OpenFlow y procesa el tráfico sin intervención continua del controlador. Se usa iperf TCP con duración larga para poder inspeccionar los flujos mientras están activos.

xterm h1:

```
iperf -s -p 5020
```

xterm h2:

```
iperf -c 200.0.0.1 -p 5020 -t 30
```

Mientras iperf corre, desde la **CLI de Mininet**:

```
sh ovs-ofctl dump-flows s1
```

Resultado esperado:

```
cookie=0x0, duration=Xs, n_packets=N, idle_timeout=60,
tcp,nw_src=192.168.1.2,nw_dst=200.0.0.1,tp_src=XXXX,tp_dst=5020
actions=mod_nw_src:200.0.0.254,mod_tp_src:10000,
mod_dl_src:00:00:00:aa:aa:aa,mod_dl_dst:00:00:00:00:00:01,output:"s1-eth1" cookie=0x0,
duration=Xs, n_packets=N, idle_timeout=60,
tcp,in_port="s1-eth1",nw_dst=200.0.0.254,tp_dst=10000
actions=mod_nw_dst:192.168.1.2,mod_tp_dst:XXXX,
mod_dl_src:00:00:00:bb:bb:bb,mod_dl_dst:00:00:00:00:00:02,output:"s1-eth2"
```

Dos flujos activos con n_packets creciendo y el controlador sin recibir ningún log nuevo. Eso confirma que el switch procesa el tráfico de forma autónoma una vez instalados los flujos.

```
mininet> sh ovs-ofctl dump-flows s1
cookie=0x0, duration=0.633s, table=0, n_packets=91682, n_bytes=4813969936, idle_timeout=60, tcp,nw_src=192.168.1.2,nw_dst=200.0.0.1,tp_src=39574,tp_dst=5020 actions=mod_nw_src:200.0.0.254,mod_tp_src:10000,mod_dl_src:00:00:00:aa:aa:aa,mod_dl_dst:00:00:00:00:00:01,output:"s1-eth1"
cookie=0x0, duration=0.592s, table=0, n_packets=68715, n_bytes=4807226, idle_timeout=60, tcp,in_port="s1-eth1",nw_dst=200.0.0.254,tp_dst=10000 actions=mod_nw_dst:192.168.1.2,mod_tp_dst:39574,mod_dl_src:00:00:00:bb:bb:bb,mod_dl_dst:00:00:00:00:00:02,output:"s1-eth2"
```

Captura: 6.7 Flujos instalados en el switch

7. Problemas Encontrados y Soluciones

ARP antes de IP: el primer paquete IP de un host privado llega cuando el controlador aún no conoce la MAC del servidor público. Se implementó la cola `_pending` y el callback `_on_mac_resolved` para reinyectar los paquetes una vez resuelta la MAC, sin perderlos.

Limpieza de entradas NAT: si los flujos expiran de forma asimétrica (por ejemplo, solo el flujo saliente genera `FlowRemoved`), puede quedar una entrada huérfana en `_nat_in`. Se mitigó usando el mismo `idle_timeout` para ambos flujos de una misma conexión, de modo que expiren aproximadamente al mismo tiempo.

IPv6: la topología deshabilita IPv6 explícitamente en todos los hosts y el switch (sysctl), evitando que el controlador reciba tráfico IPv6 inesperado que podría confundirse con paquetes IP sin procesar.

8. Conclusión

Se implementó exitosamente un NAT por puertos (PAT) sobre SDN con POX y Mininet. La solución soporta múltiples hosts privados, protocolos TCP y UDP simultáneos, resolución ARP dinámica y limpieza automática de estado al expirar los flujos. El diseño modular facilita la extensión futura (por ejemplo, timeout diferenciado por protocolo).

El paradigma SDN permite implementar lógica de red compleja (como NAT/PAT) en software puro, sin modificar el hardware de forwarding, y minimizando la intervención del controlador una vez instalados los flujos en el switch.